SECTION 1

Efficient data representations

- **Autoencoder idea:** An encoder maps inputs to a latent code and a decoder reconstructs them; reconstruction loss forces useful structure when the bottleneck or training setup prevents copying inputs trivially.
- **Undercomplete linear case:** A shallow linear autoencoder with MSE on Fashion MNIST-style inputs recovers a PCA-like projection, illustrating self-supervised learning with targets equal to inputs.
- **Constraint intuition:** Like memorizing patterned sequences instead of raw digits, limits on capacity or noise push the network to discover regularities.

SECTION 2

Stacked autoencoders

- **Deep stacks:** Symmetric encoder–decoder MLPs (e.g., 784 → 100 → 30 → 100 → 784) learn richer codes; binary cross-entropy on pixel reconstructions often trains faster than MSE for multilabel “black pixel” targets.
- **Uses:** Codes support visualization (encoder + t-SNE), unsupervised pretraining of classifiers when labels are scarce, and feature extraction on large unlabeled pools.
- **Training tricks:** Tied weights (`DenseTranspose`) halve parameters in symmetric nets; greedy layerwise stacking of shallow autoencoders was an early deep-learning pretraining recipe.

SECTION 3

Convolutional and recurrent autoencoders

- **Images:** Convolutional encoders downsample with conv/pool; decoders upsample with transpose convolutions for spatial autoencoders on Fashion MNIST.
- **Sequences:** Encoder seq2vec LSTMs and decoder vec2seq stacks (with `RepeatVector`) handle variable-length series and can treat image rows as time steps.
- **Scope:** The same unsupervised machinery extends beyond dense tabular data.

SECTION 4

Denoising and sparse autoencoders

- **Denoising:** Gaussian noise or input dropout trains recovery of clean inputs, yielding robust features and practical denoising as well as pretraining signal.
- **Sparse coding (■■):** `ActivityRegularization` encourages few active latents in an overcomplete layer.
- **KL sparsity target:** Custom regularizers match batch-mean activations to a target sparsity via KL divergence, often beating simple ■■ on coding layers.

SECTION 5

Variational autoencoders

- **Probabilistic latents:** The encoder outputs μ and log-variance; a sampling layer draws codes from a Gaussian, enabling stochastic generation after training.
- **Loss:** Reconstruction plus latent KL toward a standard normal encourages a smooth latent space where random samples decode to plausible images.
- **Semantics:** Interpolation in latent space yields blended generations (e.g., between fashion items), not mere pixel fades.

SECTION 6

Generative adversarial networks

- **Adversarial setup:** The generator maps noise to fake data; the discriminator classifies real versus fake; each training step alternates discriminator updates on mixed batches and generator updates that fool the discriminator.
- **Simple GAN loop:** Custom training (not `fit`) freezes the discriminator when updating the generator; early Fashion MNIST GANs show how quick gains can plateau.

SECTION 7

GAN training difficulties and advanced architectures

- **Failure modes:** Mode collapse, Nash equilibrium fragility, hyperparameter sensitivity, and tricks such as experience replay and minibatch discrimination aim to preserve diversity.
- **DCGAN:** Convolutional generator/discriminator guidelines (strided conv, batch norm, LeakyReLU, tanh outputs) enable more convincing images and latent arithmetic on faces.
- **Scaling quality:** Progressive growing (fade-in layers, minibatch std dev, equalized learning rate), then StyleGAN (mapping network, AdaIN, per-layer noise, style mixing) drive photorealistic synthesis—the lineage behind sites like generated portrait demos.